

第四章

微型计算机汇编语言及汇编 程序设计



本章主要内容

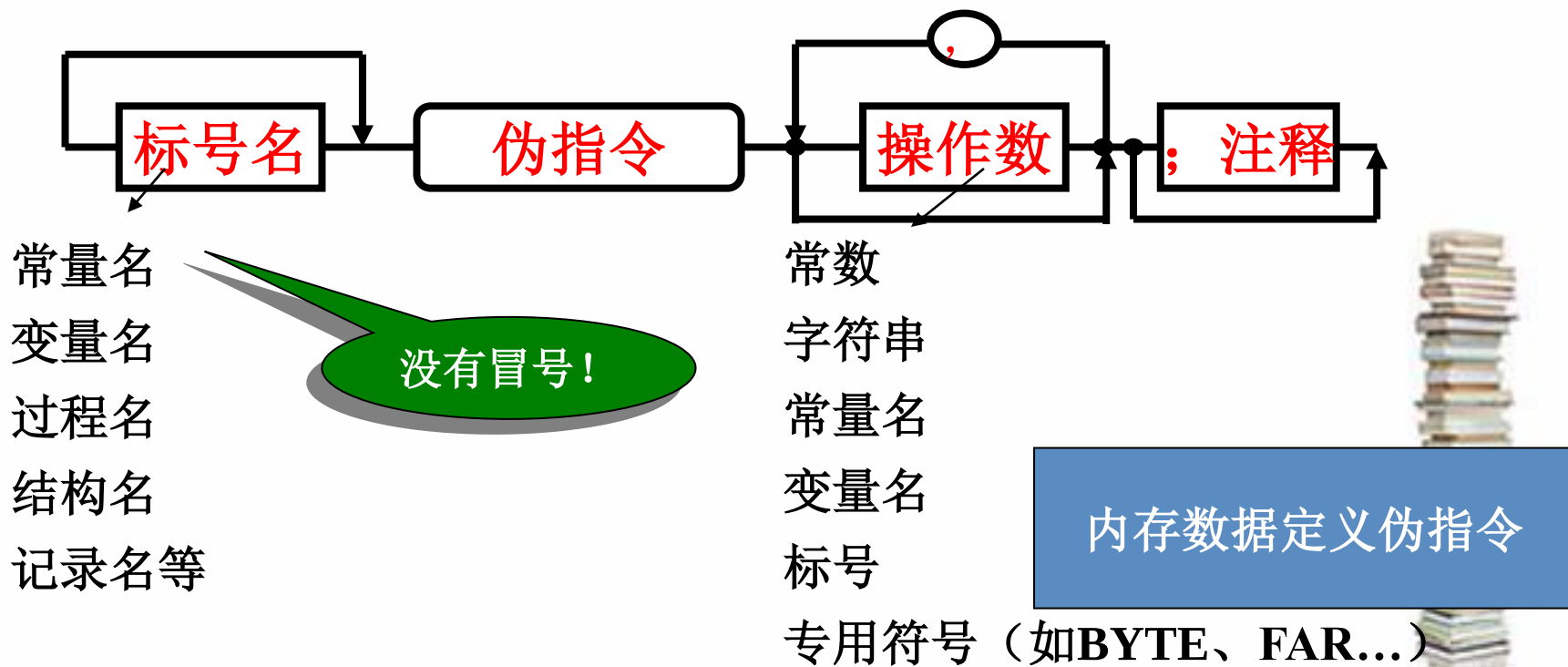
- 4.1 宏汇编语言的基本语法
- 4.2 伪指令
- 4.3 宏指令
- 4.4 系统功能调用
- 4.5 汇编程序的功能及汇编过程
- 4.6 程序设计方法



4.1 宏汇编语言的基本语法

汇编程序：用来把汇编语言编写的程序自动翻译成目的程序的软件

4.1.1 伪指令语句格式



4.1.2 常量、变量和标号

常数：没有任何属性的纯数值。在程序运行中不发生变化。

如：10010011B、0A5H、” ZHIFUCHUAN ”



标号——指令所在内存单元的符号地址

➤有三个属性：

段地址：即标号所在段的段地址；

偏移量：标号所代表存储单元的段内偏移地址；

类型：NEAR或FAR：

NEAR——表示标号所在语句与转移指令/调用指令在同一码段内,跳转时只需改变**IP**即可。

FAR——标号所在语句与转移指令/调用指令不在同一代码段内。

👁 **若没有对类型进行说明,默认为NEAR。**

➤标号通常作为**转移指令**或**CALL指令**的转移地址。



变量

变量——即内存中的存储单元或数据区。

变量名——是存储单元(数据区)的符号地址或名字。

➤变量也有三个属性：

✓**段地址**——变量所在段的段地址

✓**偏移量**——变量单元地址与段首地址之间的位移量。

✓**类型**——有**BYTE**、**WORD**和**DWORD**三种。

➤变量在程序中作为**存储器操作数**被引用。



• 标号和变量名的使用规则

- ✓ 组成：A-Z(不分大小写), 0-9, ? @ . _ \$
- ✓ 不能以数字开头，句号(.)只能作为首字符
- ✓ 长度小于31个字符
- ✓ 不能与保留字(指令助记符、伪指令、预定义符号等)重名
- ✓ 不能重复定义

例如：

正确的： LP1, AGAIN, NEXT, _GO, OK_1

错误的： 4M, LOOP, AAA, #HELP, +ONE



汇编语言的一个实例: hello.asm

data SEGMENT



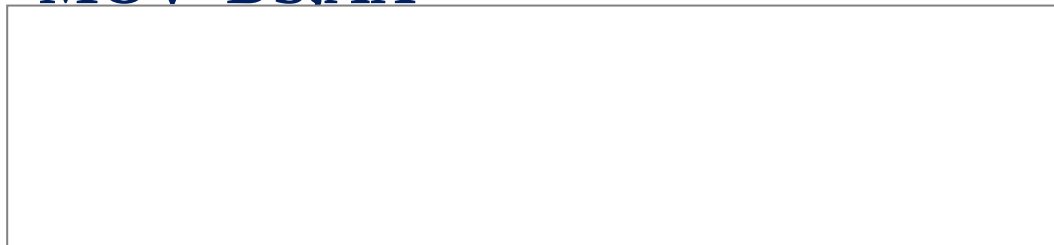
data ENDS

prog SEGMENT

ASSUME CS:*prog*,DS:*data*

start: MOV AX,*data*

MOV DS,AX



MOV AH,4CH

INT 21H

;退回DOS

prog ENDS

END *start*

变量名

ORG 100H

X DB 33H

Y DW 0A65DH

MOV AL,X

MOV BX,Y



名字

标号

4.1.3 表达式与运算符（1）

表达式： 由**操作数**和**运算符**组成的式子

(1) 算术运算符： +、-、*、/、MOD（除法取余）、SHL（左移）、SHR（右移）

例： `MOV AX, 100*4+5` ； `AX=405`

(2) 逻辑运算符： AND、OR、NOT、XOR

按位运算

例： `24H OR 0FH=2FH`

(3) 关系运算： EQ、NE、LT、LE、GT、GE

例： `MOV AX, 1234H GT 1024H`



`MOV AX, 0FFFFH`

关系运算结果总是一个数值。若关系不成立，则结果为0，若关系成立，则结果为0FFFFH

例1)算术运算符—— +、-、*、/, MOD

➤用于数字表达式:

```
MOV AX, 4*1024
```

汇编后的形式为:

```
MOV AX, 4096
```

➤用于地址表达式,例:

```
LEA SI, TAB+3
```

若TAB的偏移地址为1000H, 则汇编后的形式为:

```
LEA SI, [1003H]
```



例2)逻辑运算符—— AND、OR、XOR、NOT

逻辑运算符只能用于数字表达式中。

例：MOV CL , 36H AND 0FH

经汇编后：MOV CL , 06H

注意，不要把逻辑运算符与逻辑运算指令混淆：

例：AND AX , 3FC0H AND 0FF00H

汇编后源操作数被翻译为：3F00H，所以上述指令与AND AX, 3F00H等价。



例3)关系运算符—EQ、NE、LT、GT、LE、GE

➤关系运算的结果是一个逻辑值：**真**或**假**

关系为**真**,结果为全**1**

关系为**假**,结果为全**0**

例：MOV BX, PORT GT 300H

若PORT的值大于300H,则汇编后为：

MOV BX, 0FFFFH

否则汇编后为：

MOV BX, 0



4.1.3 表达式与运算符（2）

(4) 数值返回运算符（数值回送操作符）：

变量名

SEG（计算段基址）

OFFSET（计算偏移地址）

例1：把VARW所在段的段值送AX

MOV AX, SEG VARW

TYPE（计算符号类型值； P₁₅₇）

SIZE（计算符号名分配的字节数）（=TYPE*LENGTH）

LENGTH（计算符号名分配的数）

例3：假设VARW变量是字变量，把VARW的类型值送AL

MOV AL, TYPE VARW ; AL=2



例4)取地址运算符——SEG、OFFSET

SEG: 取变量/标号的段地址

OFFSET: 取变量/标号的偏移地址

例: VAR DB 12H

.....

MOV BX, OFFSET VAR ; 取变量VAR的偏移地址

MOV AX, SEG VAR ; 取变量VAR的段地址

注意, 以下指令的异同:

MOV BX, OFFSET VAR

LEA BX, VAR

✓OFFSET只能取静态的偏移地址;

✓LEA指令即可取静态的偏移地址, 也可取动态的偏移地址。



例5)取值运算符——TYPE、LENGTH、SIZE

- TYPE 取变量的类型 (1, 2, 4)
- LENGTH 取所定义变量的长度
(即变量中元素的个数)
- SIZE 取所定义存储区的字节数
(=TYPE*LENGTH)

例: `VAR DW 1, 2, 3, 4, 5`

则 `TYPE VAR = 2`

`LENGTH VAR = 5`

`SIZE VAR = 10`

执行: `MOV CX, SIZE VAR ; (CX)=10`



4.1.3 表达式与运算符 (3)

(5) 修改属性运算符 (属性操作符)

① 定义符号名为新类型PTR

格式: <类型> PTR <符号名>

功能: 用于指定或临时改变变量或标号的类型

例: **MOV WORD PTR [SI], 1** ; 指明字类型

② 指定新类型THIS

格式: **THIS** <类型>

功能: 为某个符号名建立新类型

例: **LABC EQU THIS BYTE**
MOV LABC, AL

执行后
[SI] = 01
[SI+1]=00

属性运算符——PTR

用来指定地址操作数的类型。

格式： <类型> **PTR** <地址操作数>

类型 $\in$ {**BYTE**, **WORD**, **DWORD**, **NEAR**, **FAR**}

◆ **BYTE**、**WORD**、**DWORD**

用于描述数据存储单元(变量)地址

◆ **NEAR**、**FAR**

用于描述转移、调用的目的地址



例6) : MOV **BYTE PTR**[DI],0 ;字节类型

MOV **WORD PTR**[DI],0 ;字类型

MOV [DI],0B5H ;类型不定

➤PTR也可用来进行强制类型转换

例: STR1 DW 1234H ;STR1定义为字类型

MOV AX,STR1 ;合法

MOV AL,STR1 ;非法

MOV AL,**BYTE PTR** STR1 ;合法



4.1.3 表达式与运算符（4）

(6) 分离运算符

①LOW 取低字节

格式：LOW <表达式>

②HIGH 取高字节

格式：HIGH <表达式>

例：设SSY=2050H

MOV AL, LOW 3080H ; AL=80H

MOV AH, HIGH SSY ; AH=20

MOV CL, LOW 3A4BH ; CL=4BH

(7) 汇编运算符的优先级

(), [], < >, LENGTH, SIZE

PTR, OFFSET,

HIGH, LOW

*, /, MOD, SHL, SHR

+, -

EQ, NE, LT, LE, GT, GE

NOT

AND

OR, XOR

SHORT



4.2 伪指令语句

1. 符号定义伪指令（赋值语句）

表达式可以是常数、数值表达式、字符串、存储器符号等

(1) 等价语句EQU

格式：<符号名> EQU <表达式>

功能：用符号来代表表达式的内容

例：COUNT EQU 100 ; 符号COUNT代表常数100
BUFF EQU 4*COUNT ; BUFF代表400
HELLO EQU "How are you!" ; HELLO代表How are you
VARW EQU THIS WORD ; 变量VARW的类型是字

注意：等价语句的符号名不能被重新赋值

1. 符号定义伪指令（赋值语句）(2)

(2) 等号语句=

数值表达式

格式：<符号名> = <表达式>

功能：用符号来代表表达式的数值

例：ABCD = 100

YY = 20+300/4

ABCD = 2*ABCD+1

可以重新赋值



例: **FIVE EQU 5**

COUNT EQU CX

TEN EQU 10

DIST = BYTE PTR[SI+BP]

GOTO = JMP

...

MOV AX, TEN

MOV CX, COUNT

ADD DIST, FIVE

DIST = WORD PTR[SI+BP+1]

ADD DIST, AX

GOTO LABEL

先定义

后引用



2. 内存数据定义伪指令（1）

(1) 字节定义语句DB

格式：[名字] DB <表达式或数据项表>

功能：表达式值或项表中的每一项是一个字节数，
它们从符号名地址开始按字节连续存放。

符号地址

例：COUNT DB 100

DB 0DH, 0AH, '\$', COUNT→

VARW DB ?, 6, ?, ?

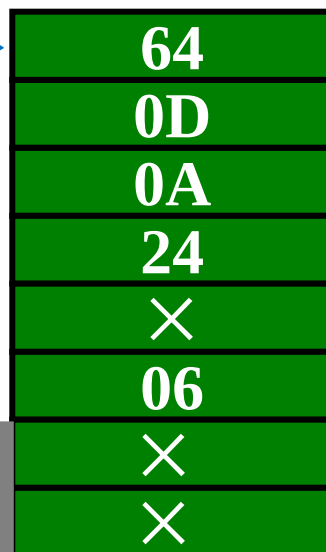
可以使用重复操作符DUP如：

BUFFER DB 8 DUP (0)

DUP可以嵌套如：

ARRAY DB 100 DUP (0, 2 DUP (1, 2) , 3)

VARW→



低

高

2. 内存数据定义伪指令（2）

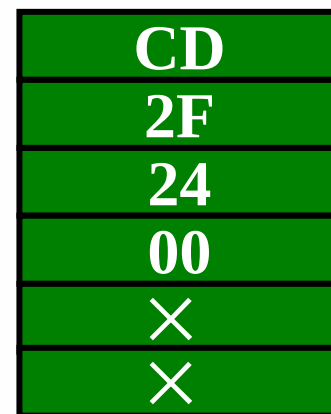
(2) 字定义语句DW

格式：[名字] DW <表达式或数据项表>

功能：表达式值或项表中的每一项是一个字数，
它们从符号名地址开始按字连续存放。

例：FLAG DW 2FCDH, 24H
DW ?

FLAG→



汇编后对应的存储区域分配情况

2. 内存数据定义伪指令（3）

(3) 四字节定义语句DD

格式：[名字] **DD** <表达式或数据项表>

功能：表达式值或项表中的每一项是四个字节数，它们从符号名地址开始按四字节从小地址连续存放。

(4) 八字节定义语句DQ

格式：[名字] **DQ** <表达式或数据项表>

功能：表达式值或项表中的每一项是八个字节数，它们从符号名地址开始按八字节从小地址连续存放。

(5) 10字节定义指令DT



例1:

DSEG1 SEGMENT

DATA_B DB 10,5,10H

DATA_W DW 100H,-4

DATA_D DD 0FFFBH

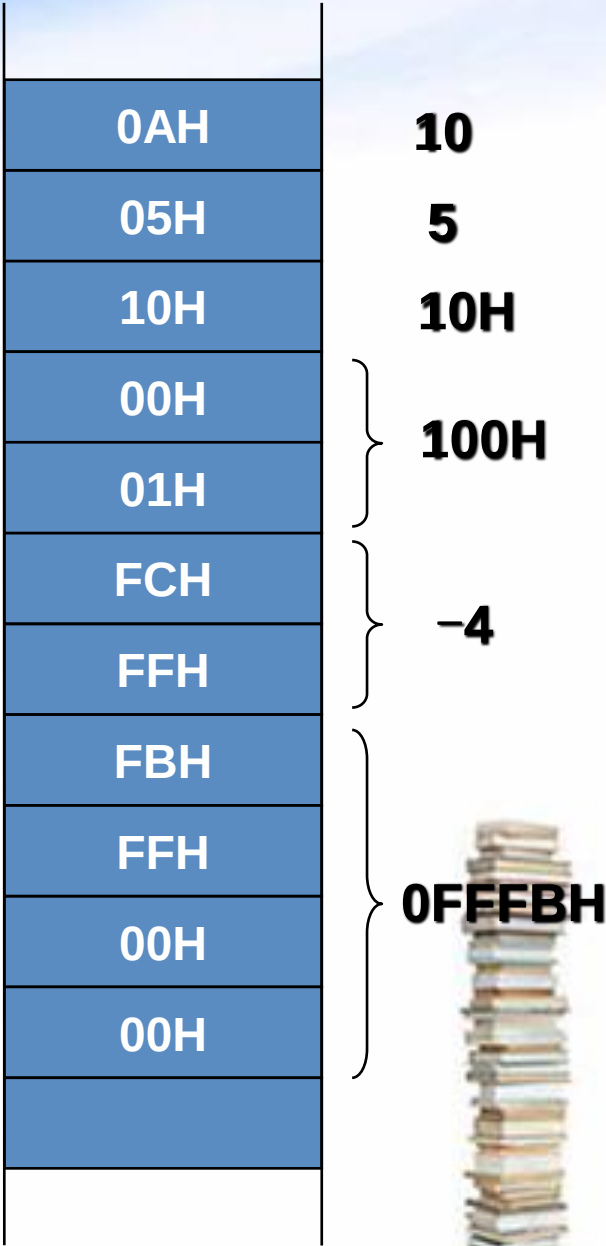
DSEG1 ENDS

汇编后的内存分配情况
如右图所示。

DSEG1: DATA_B

DATA_W

DATA_D



例2：操作数可以是字符串，例如

STR DB 'HELLO'

汇编后的情况如图：



注意下面两个定义的不同之处：

X DB 'AB' ;41H在低字节，42H在高字节

Y DW 'AB' ;42H在低字节，41H在高字节

3. 段定义伪指令（1）

段定义的一般格式：

段开始语句

<段名> SEGMENT [**定位方式**][**连接方式**][‘类型名’]

...

段结束语句

<段名> ENDS

例：一个简单的数据段

```
DSEG      SEGMENT
          BUFFER DB 8 DUP (0)
DSEG      ENDS
```

注意：

开始的段名与结束的段名必须相同，段的长度不能超过64KB，SEGMENT后面的参数是可选的

3. 段定义伪指令（2）

(1) 定位方式（定位类型）

定位类型指出了对当前段起始地址的要求，有4个选项

- PAGE** ——指定起始地址的低8位是0，即其值能被256整除（称页边界）
- PARA** ——指定起始地址的低4位为0，即其值能被16整除（称段边界）
- WORD** ——指定起始地址的最低位是0，即其值能被2整除（称为字边界）
- BYTE** ——指定起始地址是任意值

缺省定位

定位类型是用来确定程序段与程序段之间的衔接，如BYTE使得当前段紧接前一段，而PAGE可能导致最大的段间隔

3. 段定义伪指令（3）

(2) 连接方式（组合类型）

一个源程序至少有一个堆栈段

连接方式告诉连接程序本段可按某种方式连接

PUBLIC——告诉连接程序本段与其它同名同类别的段连接起来，公用一个起始点。

STACK——表示是堆栈段，连接方式同上。

隐含连接方式

NONE——表示本段不与任何段连接。

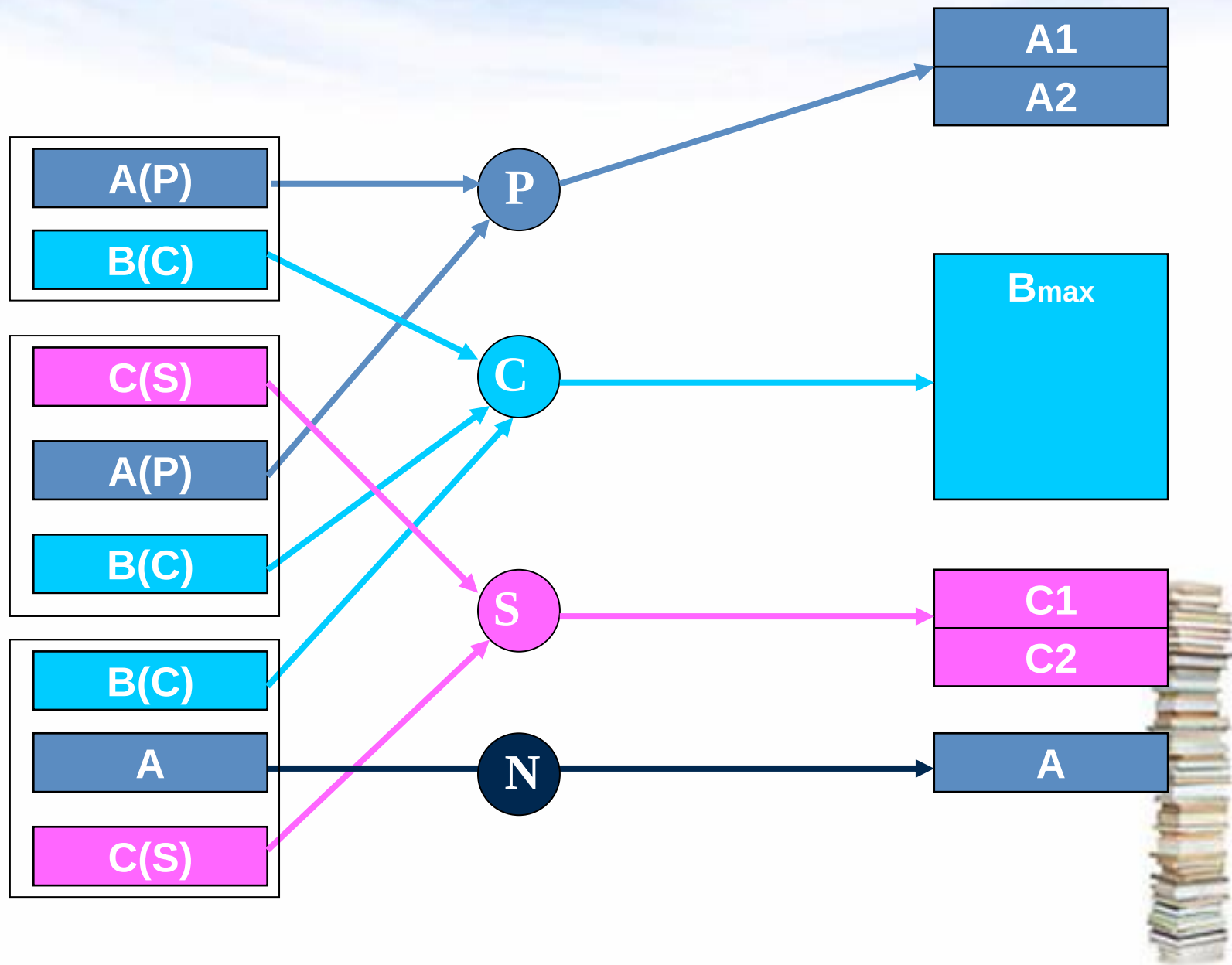
COMMON——表示本段与同名同类别的段共用同一段地址，即同名同类段相重叠。

MEMORY——本段连接时定位在所有段之上，即高地址处。

AT 表达式——表示本段定位在表达式值指定的段地址处。



- 例:连接方式



3. 段定义伪指令（4）

(3) ‘类别名’

类别名是由程序员指定的字符串，没有给出类别时，这个段的类别就为空。



```
...  
SSEG SEGMENT PARA STACK  
    DB 1024 DUP ( ? )  
SSEG ENDS  
;  
DSEG SEGMENT PARA COMMON  
MESS DB 'HELLO', 0DH, 0AH, ' $ '  
DSEG ENDS  
;
```

...
CSEG SEGMENT **PARA** PUBLIC
...
CSEG ENDS
END START



4. 段寄存器说明伪指令

格式: **ASSUME** 段寄存器: 段定义名1[, 段寄存器: 段定义名2, ...]

功能: 告诉汇编程序在汇编时, 段寄存器的符号基址

例:

DSEG1 SEGMENT VARW DW 12 ... DSEG1 ENDS ; DSEG2 SEGMENT XXX DW 0 YYY DW 0 DSEG2 ENDS ;	...	CSEG SEGMENT ASSUME CS: CSEG, DS: DSEG1, ES: DSEG2 MOV AX, DSEG1 MOV DS , AX MOV AX, DSEG2 MOV ES, AX <此处加入你自己的程序段> CSEG ENDS ...
---	-----	---



ASSUME伪指令

- 在代码段中，还必须明确段和段寄存器的关系，这由ASSUME语句来指定。如

ASSUME CS:code, DS:data, ES:data

语句中的code和data为段名。

这个语句说明：

1. CS将指向名字为code的代码段
2. DS和ES将指向名字为data的数据段



但要注意,ASSUME伪指令只是告知汇编程序有关段寄存器与段的关系,并没有给段寄存器赋予实际的初值。故下面的语句

```
MOV    AX,DATA
```

```
MOV    DS,AX
```

```
MOV    ES,AX
```

将段基址装入段寄存器。如果程序中用到堆栈段,则SS也需装入实际的初值。

代码段基址不需要程序员装入CS寄存器,而由DOS负责装入。



5. 过程（子程序）定义伪指令

格式：<过程名> **PROC** [过程属性]

.....

.....

<过程名> **ENDP**

段间为FAR
段内为NEAR



例1: 调用程序和子程序在同一代码段中

XXX SEGMENT

main proc far

.....

call subr1

.....

ret

main endp

subr1 proc near

.....

ret

subr1 endp

.....

XXX ENDS

XXX SEGMENT

main proc far

.....

call subr1

.....

ret

subr1 proc near

.....

ret

subr1 endp

main endp

.....

XXX ENDS

一个过程

一个过程



例2:
调用过程和
子过程不在
同一个代码
段内

SEGX SEGMENT

.....

subt **proc** **far**

.....

ret

subt **endp**

.....

call **subt**

.....

SEGX ENDS

SEGY SEGMENT

.....

call **subt**

.....

SEGY ENDS

所以过程属性
为FAR

在同一段中调用
也在不同段中调用



6. 模块开始伪指令

格式: **NAME** 模块名

7. 模块结束伪指令

格式: **END** [启动标号或过程名]

功能: 告诉汇编程序源文件结束, 并给出执行程序的入口地址



8. 定位伪指令

格式：ORG <表达式>

功能：把以下语句定义的内存数据或程序，从表达式指定的起点开始连续存放。

例： ABC SEGMENT
 ORG 100H
 begin: ...
 ...
 ABC ENDS



4.3 宏指令

宏指令：为了简化汇编语言源程序的书写提供的指令

宏指令的使用要经过三个步骤：

- (1) 宏定义：对各个宏指令进行定义，并分别起一个名字
- (2) 宏调用：在需要的地方，通过宏指令名来调用它
- (3) 宏展开：由汇编程序用宏定义中的指令替代宏指令名

由汇编程序
完成的



1. 宏代换语句

格式：宏名 **MACRO** [形式参数1, 形式参数2, ...]

...
...
ENDM }

功能：为指令序列定义一个宏名，称宏指令

形式参数是任选项，可以用来代换宏体中的某些参数或符号。需要代换的**符号**前，必须加一个宏代换符&

例：AA **MACRO** Y, X
 MOV CL, X
 RO&Y AX, CL
 ENDM

需要替换
的符号

调用时：

...
AA R, 5
...

实在参数

象指令一样出
现在程序中

2. 局域符号定义语句

格式: **LOCAL** 符号1[, 符号2, ...]

功能: 指出符号仅在宏代换中有意义

(目的是避免在宏展开时引起符号重复)

例: **AA MACRO**

LOCAL LP1

...

LP1 : ADD AL, 30H

...

ENDM



3. 删除宏定义语句

格式: **PURGE** 宏定义名1[, 宏定义名2, ...]

功能: 告诉汇编程序取消某些宏。使之在宏调用的地方不再生成宏展开。

4. 退出宏定义

格式: **EXITM**

功能: 在宏体或重复定义的语句中, 遇到**EXITM**语句时, 终止以后的宏展开。

一般与条件伪指令
一起配合使用



5. 重复定义语句

格式1: REPT <表达式>
...
ENDM

表达式是重复次数

格式2: IRP 形式参数, <参数1, 参数2, ...>
...
ENDM

参数个数是重复次数, 重复一次将一个参数代入形式参数中

格式3: IRPC 形式参数, 字符串
...
ENDM

字符串中字符数是重复次数, 重复一次代入一个字符到形式参数

4.4 系统功能调用

调用方法：

- (1) 准备入口参数
- (2) 把功能调用号送AH寄存器
- (3) 发软中断指令 **INT 21H**



1. 键盘输入单字符——01H号功能调用

功能：从键盘读一字符，并将该字符回显在显示器上

入口参数：无

出口参数：AL=读到字符的ASCII码

说明：如果读到的字符是Ctrl+c或Ctrl+Break则结束程序

例：

```
...  
MOV  AH, 1  
INT  21H  
...
```



2. 键盘输入字符串——0AH号功能调用

功 能：从键盘读一字符串。

入口参数：

DS: **DX=缓冲区首址**，缓冲区的第一个字节放缓冲区最大容量数

出口参数：接收到的字符串放在缓冲区中第三个字节开始

说 明：

- ① 缓冲区第二个字节存放实际读入的字符数。
- ② 字符串以回车结束(0DH)
- ③ 如果输入字符串超过了缓冲的区容量，超出的部分丢失，同时响铃
- ④ 如果在输入时按Ctrl+C或Ctrl+Break，则结束程序

例：

```
...  
BUF DB 20 ;缓冲区最大容量数  
DB ? 存放实际读入的字符数  
DB 20 DUP ( ? )
```

```
MOV DX, OFFSET BUF  
MOV AH, 0AH  
INT 21H ;输入字符串
```

...



3. 输出单字符——02H号功能调用

功能：寄存器DL中的字符从屏幕上显示输出

入口参数：DL=要输出字符的ASCII码

出口参数：无

例：

...

MOV DL, 'A'

MOV AH, 2

INT 21H

...



4. 输出字符串——09H号功能调用

功 能：将指定缓冲区中的字符串从屏幕上显示输出
入口参数：DS: **DX**=要输出字符的首址，字符串以字符
 \$ 为结束标志
出口参数：无

例：

```
...  
BUF DB 'good bye $'  
  
...  
MOV DX, OFFSET BUF  
MOV AH, 09H  
INT 21H ;字符串从屏幕上显示  
...
```



5. 直接控制台输入/输出单字符——06H号 功能调用

功 能：直接键盘输入屏幕显示输出

入口参数：DL=OFFH，表示输入；否则表示输出，
 DL=输出字符代码

出口参数：输入时，AL=输入字符代码，输出时，无

例：

...

MOV DL, 0FFH

MOV AH, 6

INT 21H

MOV DL, 'a'

} 键盘输入单字符送AL

说明：1、在输入时，如无字符可读，**不等待**
2、不检查Ctrl+C或Ctrl+Break键
3、在读到字符时也不回显

6. 无回显直接控制台输入单字符——07H号功能调用

功 能：从键盘上读一个字符到AL

入口参数：无

出口参数：AL=读到字符的代码

不检查是否为Ctrl+Break键

7. 无回显键盘输入单字符——08H号功能调用

功 能：从键盘上读一个字符到AL

入口参数：无

出口参数：AL=读到字符的代码

检查是否为Ctrl+Break键



8. 从串口输入单字符——03H号功能调用

功 能：从异步通信口串行输入的字符到AL中

入口参数：无

出口参数：AL=读到字符的代码

9. 向串口输出单字符——04H号功能调用

功 能：将DL寄存器中的字符通过异步通信口串行输出

入口参数：DL=要输出字符的代码

出口参数：无



10. 设置日期——2BH号功能调用

功 能： 设置有效日期

入口参数： CX=年号， ， DH=月号， DL=日期
都是装配型BCD码

出口参数： AL=0， 表示设置成功； 否则AL=0FFH

例：

```
...  
MOV    CX, 2008H  
MOV    DH, 04H  
MOV    DL, 14H  
MOV    AH, 2BH  
INT    21H  
...
```



11. 取得日期——2AH号功能调用

功 能：将当前日期取到CX和DX寄存器中

入口参数：无

出口参数：CX=年号，DH=月号，DL=日期
都是装配型BCD码

12. 设置时间——2DH号功能调用

功 能：设置有效时间

入口参数：CH=时数，CL=分数，DH=秒数，
DL=百分之一秒数，都是装配型BCD码

出口参数：AL=0，表示设置成功；否则AL=0FFH



13. 取得时间——2CH号功能调用

功 能：将当前时间取到CX和DX寄存器中

入口参数：无

出口参数：CX=时分，DH=秒，都是装配型BCD码

例： ...
 MOV AH, 2CH
 INT 21H
 ...



14. 返回操作系统——4CH号功能调用

功 能：结束当前正在执行的程序，并返回操作系统
入口参数：无
出口参数：无

回到DOS提示符下C: >

例： ...
 MOV AH, 4CH
 INT 21H
 ...



- (一)基本的汇编语言程序框架如下：

```
stack  SEGMENT PARA 'stack'  
        DB  100 DUP('stack')
```

```
stack  ENDS
```

```
data    SEGMENT
```

```
        <数据、变量在此定义>
```

```
data    ENDS
```

```
code    SEGMENT
```

```
        ASSUME CS:code, DS:data, ES:data
```

```
start:  MOV  AX, data
```

```
        MOV  DS, AX
```

```
        MOV  ES, AX
```

```
        MOV  AX, stack
```

```
        MOV  SS, AX
```

```
        <此处加入你自己的程序段>
```

```
        MOV  AL, 4CH
```

```
        INT  21H
```

```
code    ENDS
```

```
        END  start
```

堆栈段

数据段

代码段



例1:

从键盘接收并显示一个字符串，回车，换行后再次显示输入的字符

Stack segment stack

; 堆栈段定义

stack ends

; 数据段定义

data segment

OBUF DB '>', 0DH, 0AH, '\$'

IBUF DB 0FFH, 0, 255 DUP(?)

data ends

; 代码段定义

code s

; 段声明

assume ds:data,cs:code,ss:stack

; 过程定义

begin proc far

start: push ds

sub ax,ax

push ax

mov ax,data

mov ds,ax

MOV DX, OFFSET OBUF

MOV AH,9

INT 21H

; 显示提示符

MOV DX, OFFSET IBUF

MOV AH,10

INT 21H ; 键入字符串

MOV BL, IBUF+1

; BX=实际字符串长

MOV BH, 0

MOV IBUF[BX+2], '\$'

MOV DL,0AH

MOV AH, 2

INT 21H ; 换行

MOV DX, OFFSET IBUF+2

MOV AH, 9

INT 21H ; 再次显示字符串

ret

; 主子程序返回

Begin endp

code ends

end **start**

; 源程序结束



汇编程序编程应注意的问题

- 任何一个源程序必须至少有一个代码段，其它段视程序要求而定。
- 段寄存器的装填（DS、ES、SS由用户自行装填，CS不须用户处理）。

方法： **MOV AX, ******

MOV DS, AX (或ES、SS)

- DOS下汇编程序的正常结束

方法1： **MOV AH, 4CH**

INT 21H

方法2： 程序名： **PROC FAR**

PUSH DS

MOV AX, 0

PUSH AX

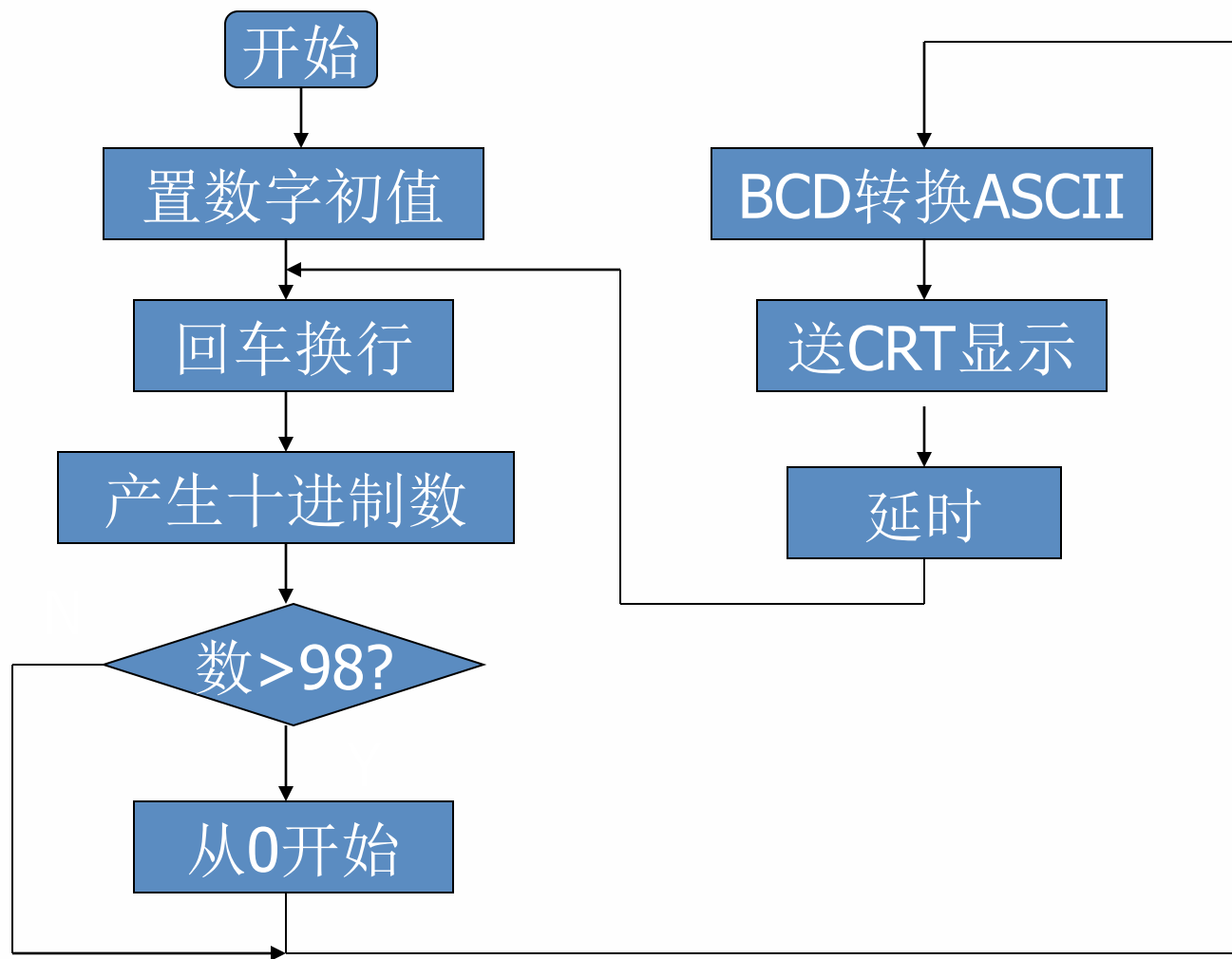
.....

RET

程序名 **ENDP**



例2：在CRT上循环显示0-98之间的十进制数。



```

DATA    SEGMENT
BUF     DB   3 DUP ( ? )
DATA    ENDS

```

```

CODE    SEGMENT

        ASSUME    CS:CODE,
DS:DATA

MAIN    PROC      FAR

        PUSH      DS

        MOV        AX,0

        PUSH      AX

        MOV AX , DATA

        MOV DS , AX

        MOV        BL,-1

        PUSH      BX      ; 保护BL

NEXT :   MOV        SI,OFFSET BUF

        MOV        DL, 0DH ; 回车符

        MOV        AH,2

        INT        21H ; 送显示

```

```

        mov  dl,0ah ;输出换行符

        mov  ah,2

        int  21h

        pop  bx      ;取出bl

        mov  al, bl   ; 产生十进制数

        inc  al

        daa

        cmp  al,99h

        jc   next1   ; al低于99H转Next1

        mov  al, 0

Next1:   mov  bl,al

        push bx      ;暂存bl(十进制数)

        mov  dl,al   ;十进制数转ASCII

        mov  cl,4     ;高4位转ASCII

        shr  al,cl

        or   al,30h

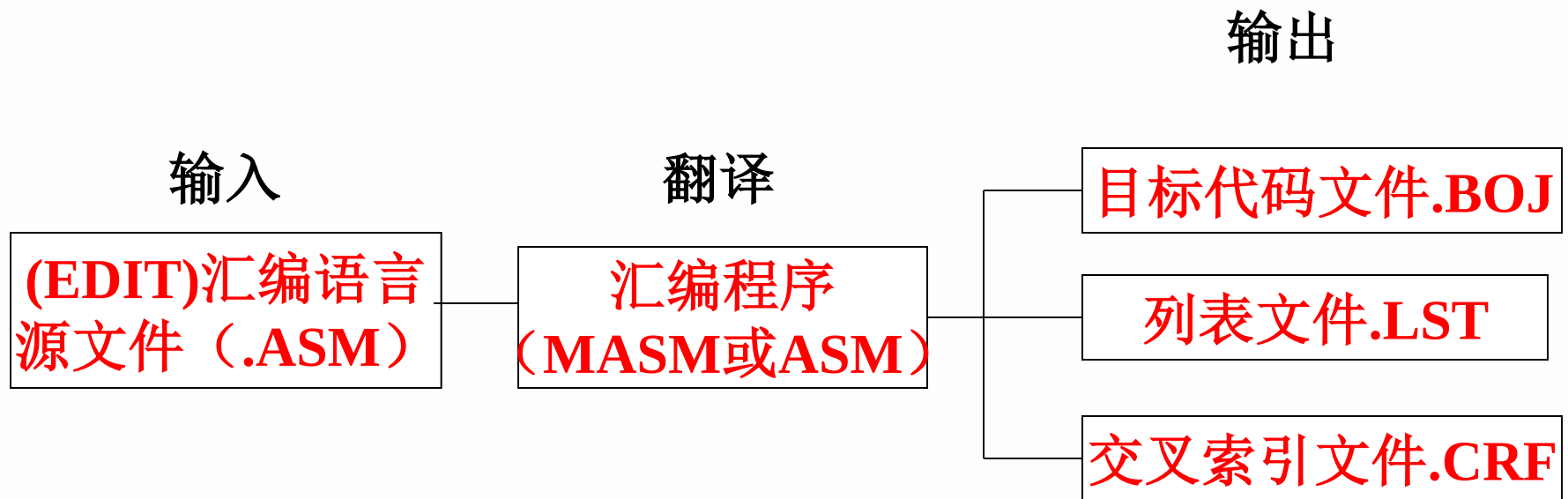
        mov  [si],al

```

```
    inc    si
    mov    al,dl                ; 低4位转ASCII
    and    al,0fh
    or     al,30h
    mov    [si],al
    inc    si
    mov    al,'$'
    mov    [si],al             ; 字符串结束符
    mov    dx,offset buf       ; 9号调用送显示
    mov    ah,9
    int     21h
    mov    cx,0ffffh           ; 延时
Dy: loop   dy
    jmp    next
    ret
Main endp
code ends
    end    main
```



4.5.1 汇编程序的功能



汇编程序的功能就是把用助记符书写的汇编语言源程序，翻译成机器语言目标代码；同时进行语法检查，指出语法错误所在行的行号。

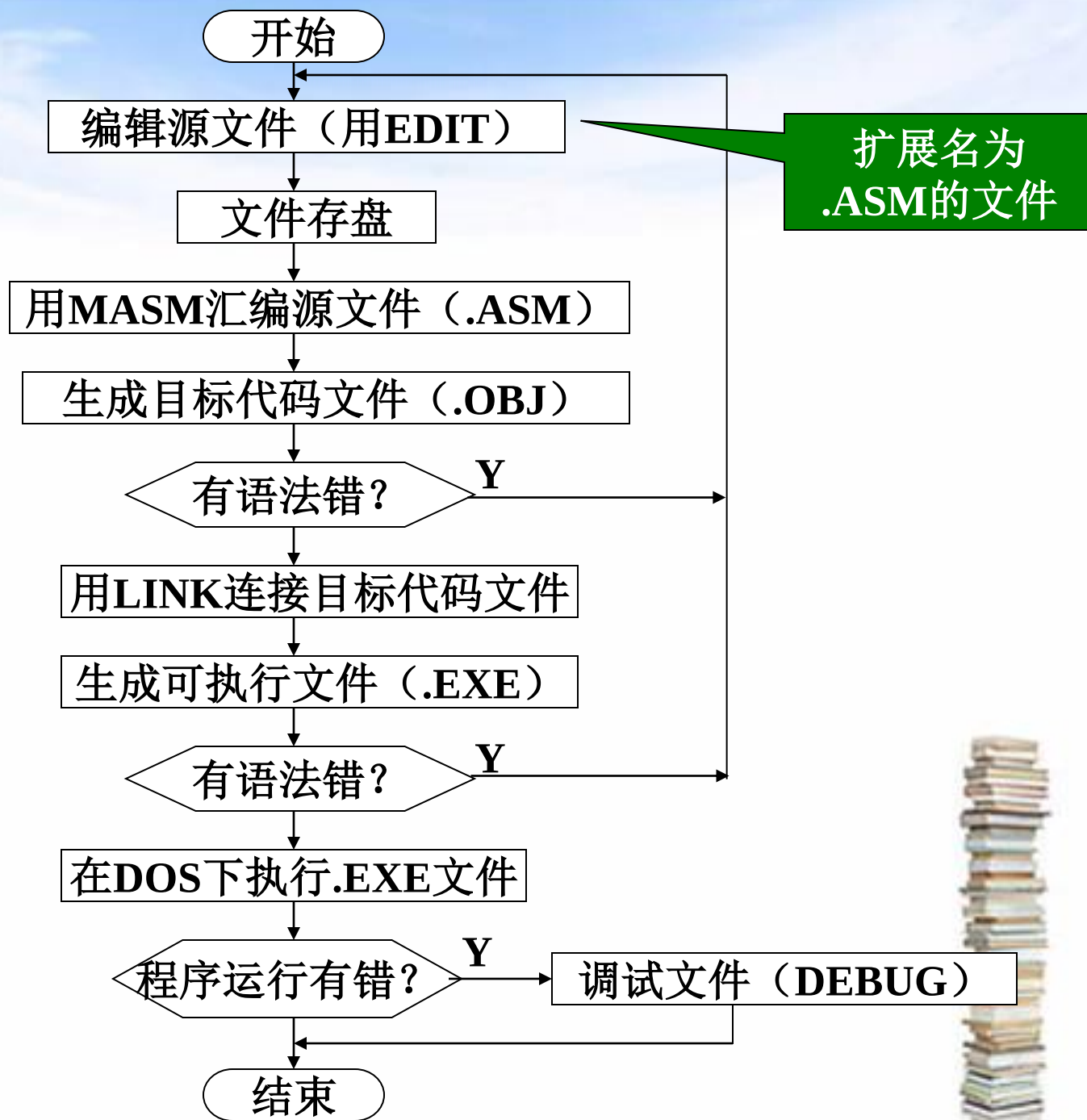
在计算机上运行汇编语言程序的步骤

- 用编辑程序建立XX.ASM程序 文本编辑EIDT 汇编程序MASM
- 用汇编程序把XX.ASM程序转换成XX.OBJ程序 连接程序LINK
- 用连接程序把XX.OBJ程序转换成XX.EXE
- 用调试程序调试程序 XX.EXE DEBUG
- 用DOS命令直接键入程序名执行



7.5.2

程序的编辑、汇编及连接



上机步骤:

1. 编辑程序

在DOS方式下

EDIT 文件名.ASM 回车
输入程序
存盘

缺省方式，以源文件名为目标文件名，无列表文件；无交叉索引文件

2. 汇编程序

MASM 文件名； 回车

如果有语法错误，用EDIT修改文件

3. 程序连接

LINK 文件名； 回车

4. 执行程序

文件名 回车

如果执行程序有错，进入第5步

5. 调试程序

DEBUG 文件名.EXE 回车

进入了DEBUG
调试状态



7.6

构造一个源程序的基本格式:

```
DATA    SEGMENT
        ...           ; 存放数据项的数据段
DATA    ENDS
EXTRA   SEGMENT
        ...           ; 存放数据项的附加段
EXTRA   ENDS
STAC    SEGMENT PARA STACK
        ...           ; 作堆栈用的堆栈段
STAC    ENDS
COSEG   SEGMENT
        ASSUME CS: COSEG, DS: DATA
        ASSUME SS: STAC, ES: EXTRA
BEING:  MOV AX, DATA
        MOV DS, AX
        ...           ; 存放指令序列
COSEG   ENDS
        END BEING
```



•DEBUG程序命令（1）

1. A命令（汇编命令）

格式：A [地址]

功能：从指定地址把宏汇编语言语句直接汇编入内存

2. D命令（显示命令）

格式：D [地址] 或 D [范围]

功能：从指定地址开始显示内存中80H字节的内容，
或显示指定范围的内容

说明：

地址：由段值和偏移两部分组成。如 DS: 100，段值可以省略

范围：有两种表示方法，一种是用起始地址和结束地址如DS: 100 110，
另一种用起始地址和长度表示。如DS: 100 L10

•DEBUG程序命令（2）

3. G命令（执行命令）

格式：G [=起始地址][断点地址表]

功能：从当前地址或指定地址执行遇到断点停止执行

4. R命令（寄存器命令）

格式：R ; 显示所有寄存器的内容

R 寄存器名 ; 显示并可修改寄存器的内容

RF ; 显示并可修改标志寄存器的内容

5. Q命令（停止命令）

格式：Q

功能：退出DEBUG程序，不保存内存中调试的文件



•DEBUG程序命令（3）

6. T命令（跟踪命令）

格式：T [=地址][数值]

功能：从当前地址或指定地址执行一条或指定的条数指令并显示各寄存器的内容及下一条要执行的指令

7. U命令（反汇编命令）

格式：U [范围] 或 U [地址]

功能：把指定范围的内容反汇编或从指定地址开始反汇编16条



DEBUG程序命令（4）

8. E命令（修改内存单元命令）

格式：E [地址] [表]

功能：用表中的内容替换内存中一个或多个字节里的内容
（表中的数据以空格分隔，一个数据代表一个单元的内容）

例：把当前数据段，100H开始的连续单元分别修改为
11H，22H，33H，44H，55H

E 100 11 22 33 44 55



DEBUG中标志位的符号表示

标志名	标志为 1	标志为 0
OF 溢出（是/否）	OV	NV
DF 方向（减量/增量）	DN	UP
IF 中断（允许/关闭）	EI	DI
SF 符号（负/正）	NG	PL
ZF 零（是/否）	ZR	NZ
AF 辅助进位（是/否）	AC	NA
PF 奇偶（偶/奇）	PE	PO
CF 进位（是/否）	CY	NC

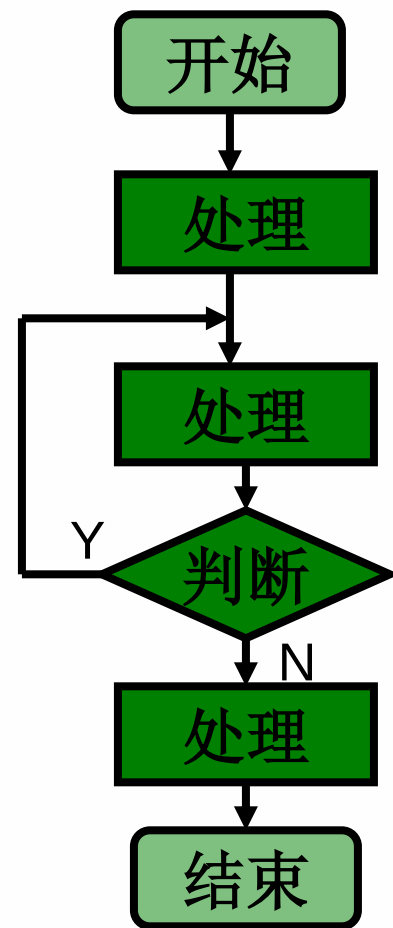


4.6 程序设计步骤

程序：计算机命令（语句）的有序集合。

汇编语言程序设计步骤：

- (1) 分析问题
- (2) 建立数学模型
- (3) 确定算法
- (4) 绘制程序流程图
- (5) 内存空间分配
- (6) 编制程序与静态检查
- (7) 程序调试



4.6.1 简单程序（1）

例：将内存单元**10050H**的内容拆成两段，每段**4**位，分别存入**10051H**（低4位）和**10052H**（高4位）单元。即**10051H**和**10052H**单元的高4位都为0。

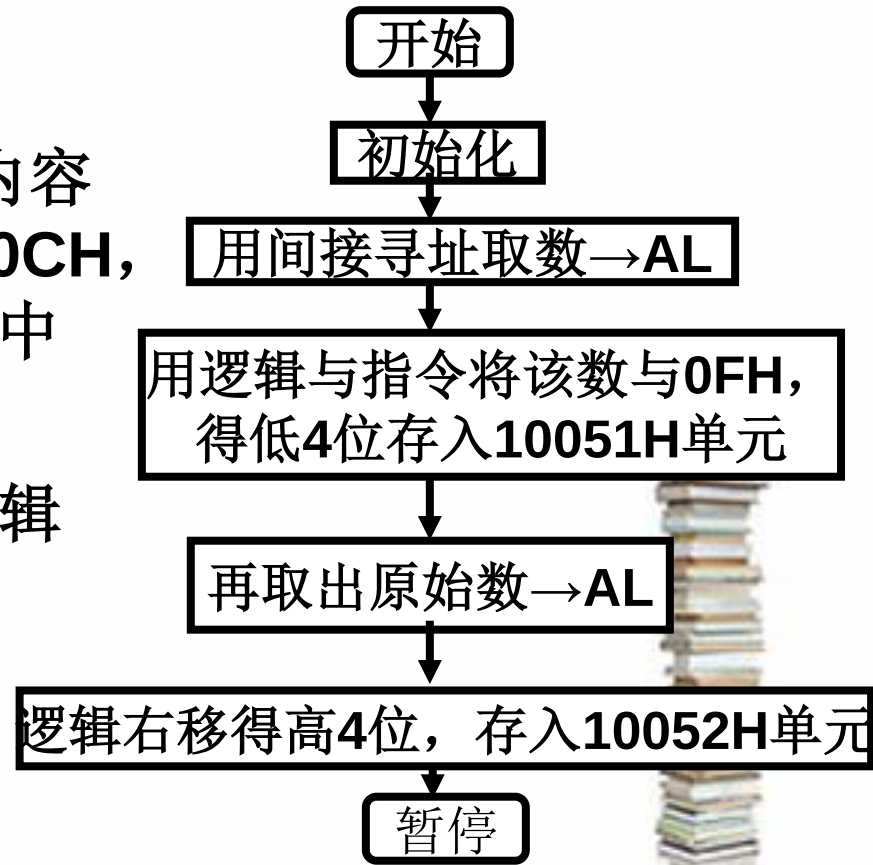
①分析：

理解题意，假设**10050H**单元的内容是**6CH**，题意是把它分成**06H**和**0CH**，分别存入**10051H**和**10052H**单元中

②确定算法：

可以通过移位指令，也可以用逻辑与运算指令（与**0FH**）

③绘制程序流程图



4.2 简单程序（2）

④内存分配

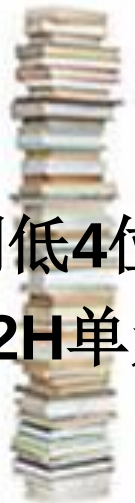
内存地址	内容
10050H	原始数(6CH)
10051H	原低4位(0CH)
10052H	原高4位(06H)

简单地画一个表，便于日后查看

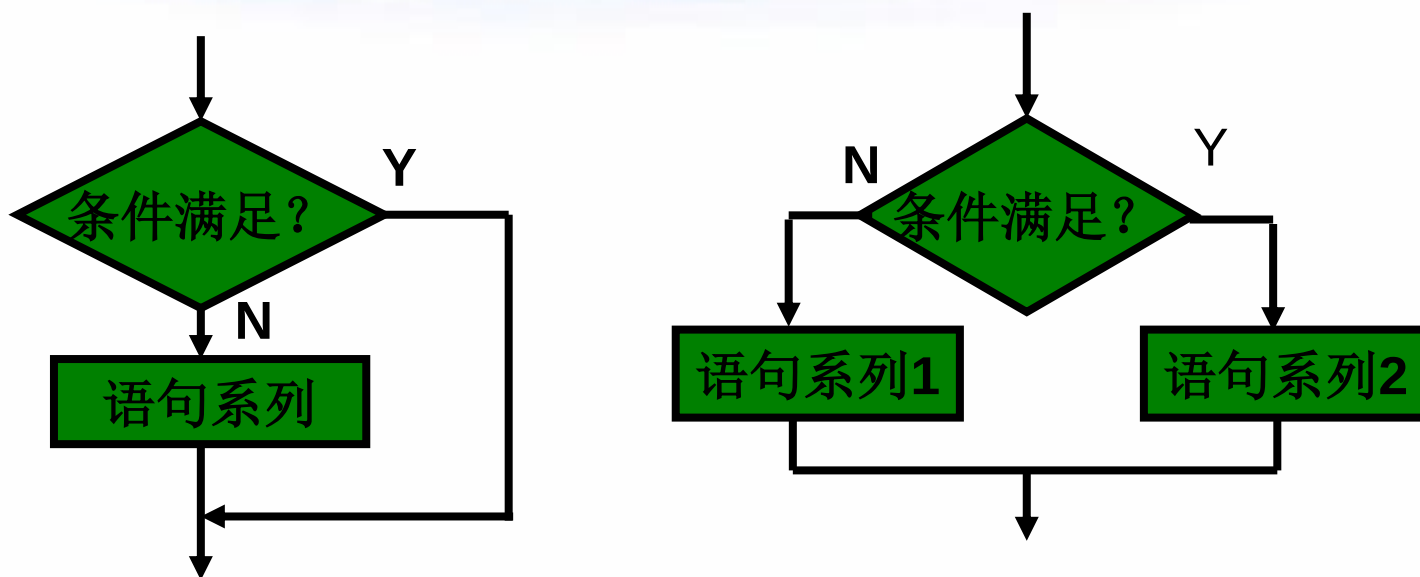
⑥静态检查程序是否有错

⑤ 编制程序

```
MOV  AX, 1000H
MOV  DS, AX      ; DS=1000H
MOV  SI, 50H     ; 源指针SI=50H
MOV  AL, [SI]    ; 读10050H单元
AND  AL, 0FH     ; AL高4位清0
MOV  [SI+1], AL
MOV  AL, [SI]
MOV  CL, 4
SHR  AL, CL      ; 高4位移到低4位
MOV  [SI+2], AL; 存入10052H单元
...
```



4.6.2 分支程序（1）



两种分支结构示意图

例：编一个程序。从外设71H取一个数M，判断其值是否在10和20之间，即 $10 \leq M < 20$ 。如果 $M \geq 20$ ，则送0FFH给外设73H；如果 $M < 10$ ，则送00H给外设73H；如果 $10 \leq M < 20$ ，则送88H给外设73H。



4.6.2 分支程序 (2)

③ 画流程图

① 分析

根据题意需要两次判断M的大小，根据判断结果把00H或0FFH或88H输出到73H端口。

② 确定算法

可以使用低于JB和高于或等于转移指令JAE

...

IN AL, 71H; 71H端口数→AL

CLC

CMP AL, 10

JB LP1 ; 小于10转LP1

CMP AL, 20

JAE LP2 ; 大于或等于20转LP2

MOV BL, 88H

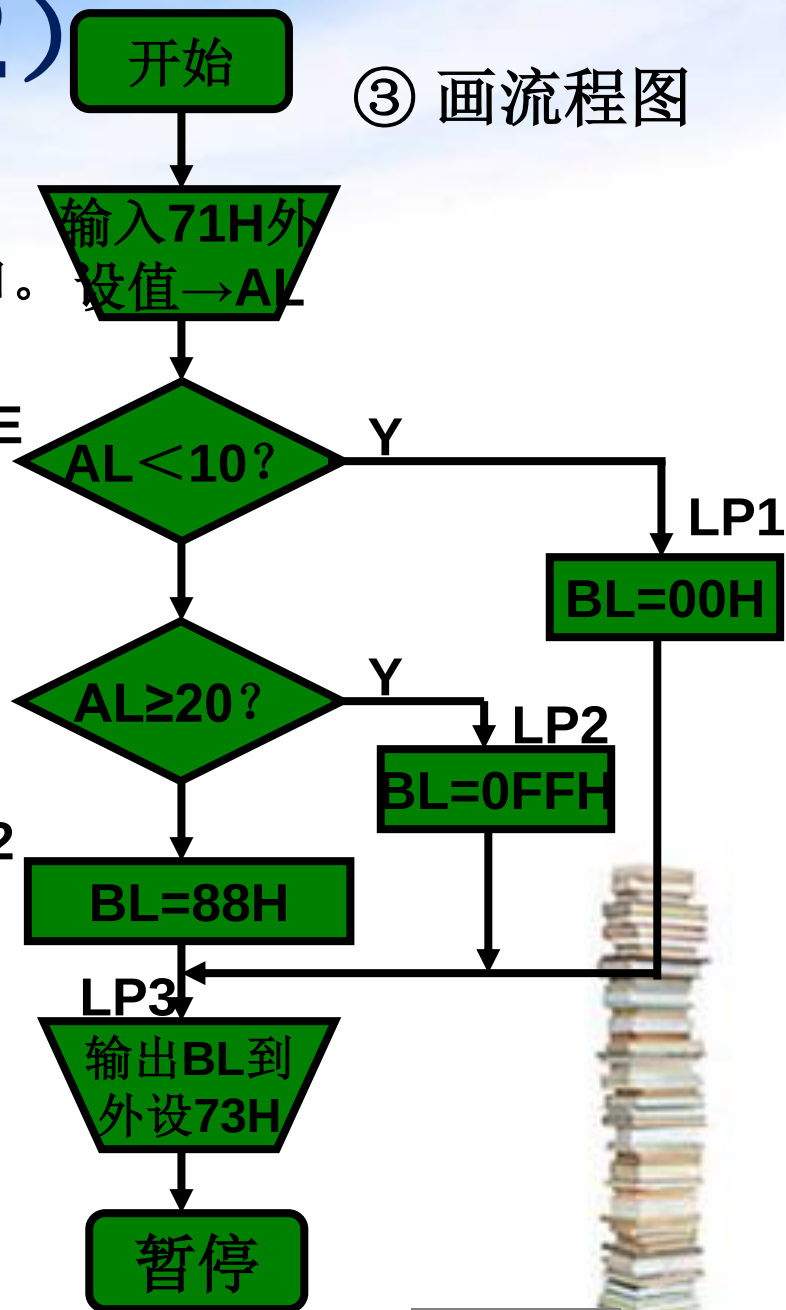
JMP LP3

LP1: MOV BL, 00H

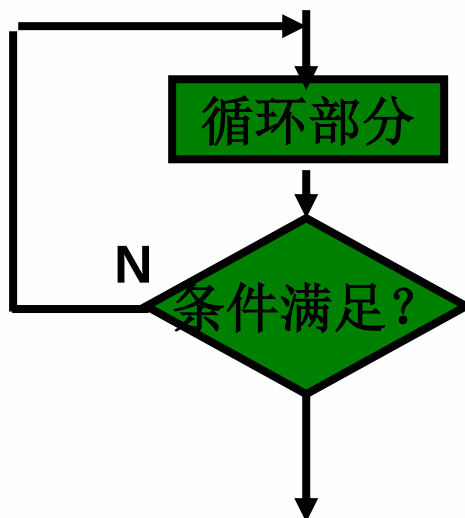
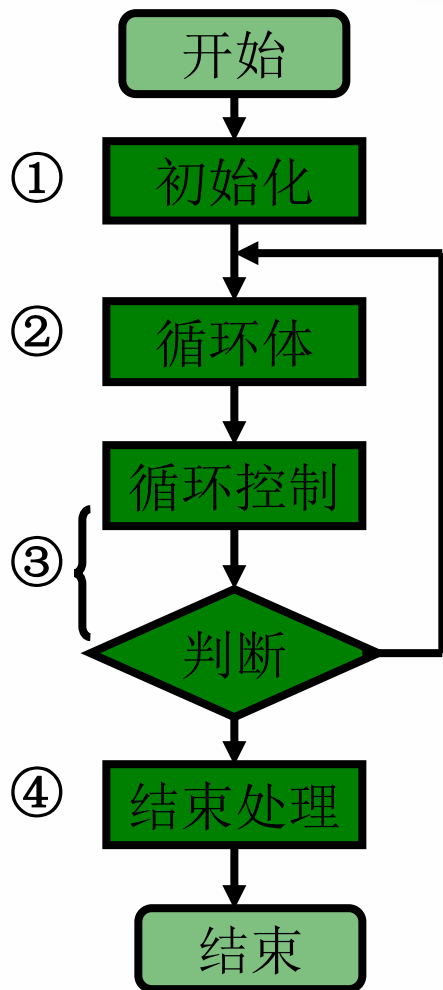
JMP LP3

LP2: MOV BL, 0FFH

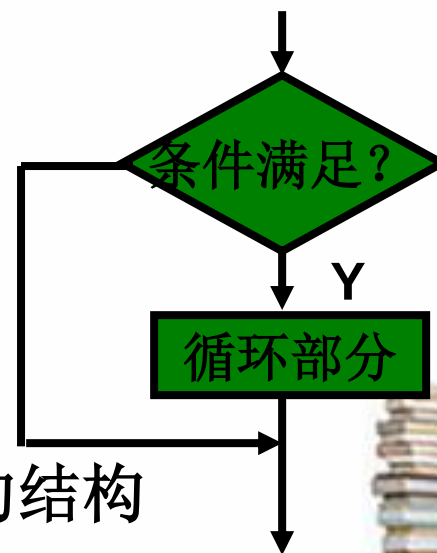
LP3: MOV AL, BL ; 将BL的内容输出到端口73H
OUT 73H, AL



4.6.3 循环程序



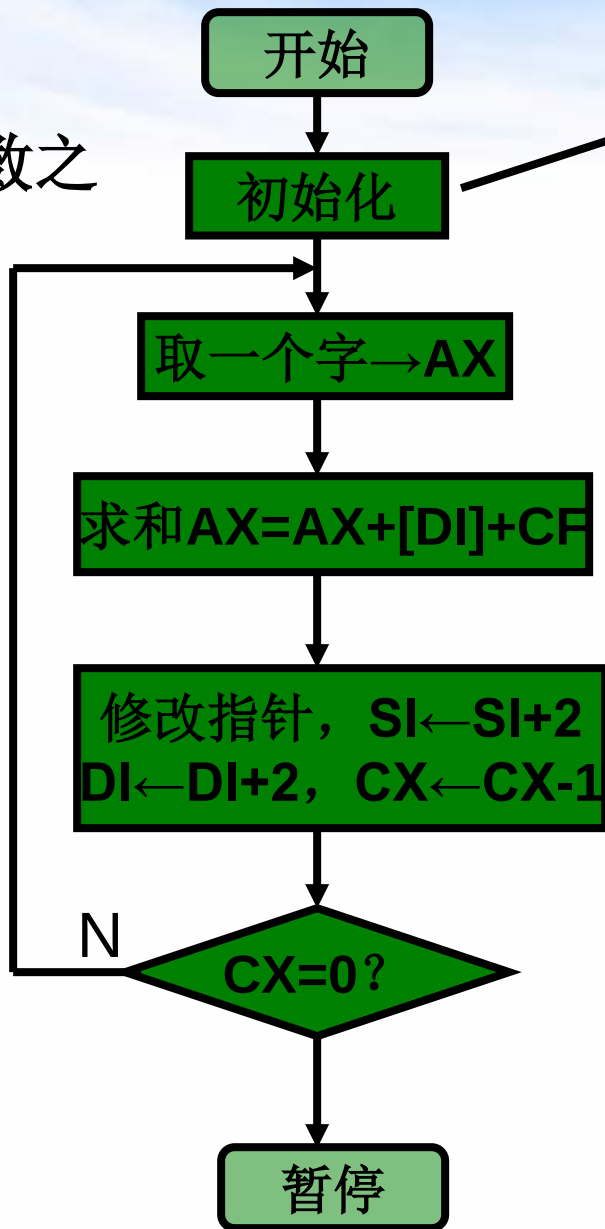
先执行循环体后判断的结构



先判断后执行循环体的结构

•循环程序举例

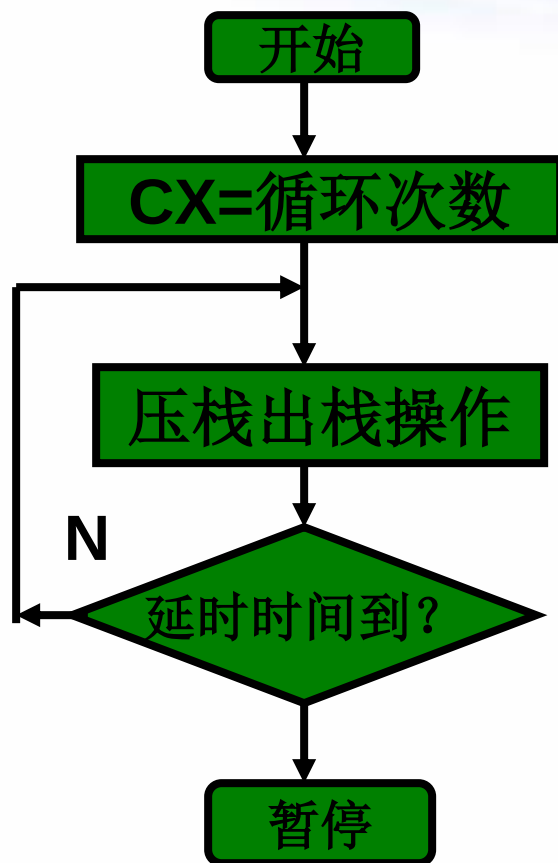
例：
求两个多字节数之和。这两个数在10050H开始的单元中，连续存放，低位在小地址一端，结果放在两个数之后。设这两个多字节数均为八个字节长。



```
MOV    AX, 1000H
MOV    DS, AX
MOV    SI, 50H
MOV    DI, 58H
MOV    BX, 60H
MOV    CX, 4
CLC
AA:    MOV    AX, [SI]
        ADC    AX, [DI]
        MOV    [BX], AX
        PUSHF
        ADD    SI, 2
        ADD    DI, 2
        ADD    BX, 2
        POPF
        LOOP   AA
        HLT
```



例：设计一个软件延时程序，延时时间约1ms



```

MOV BX, 1000
LP2: MOV CX, 176H ; 循环374次
LP1: PUSHF
      POPF
      LOOP LP1      ; CX-1, CX≠0转LP1
      DEC BX
      JNZ LP2
      HLT
  
```

PUSHF	10节拍	}	21.4节拍	假设时钟为8M
POPF	8节拍			
LOOP	3.4节拍			

$$\text{循环次数} = \frac{\text{延时时间}}{\text{一次循环时间}} = \frac{1\text{ms}}{21.4 \times 0.125\mu\text{s}} \approx 374\text{次}$$



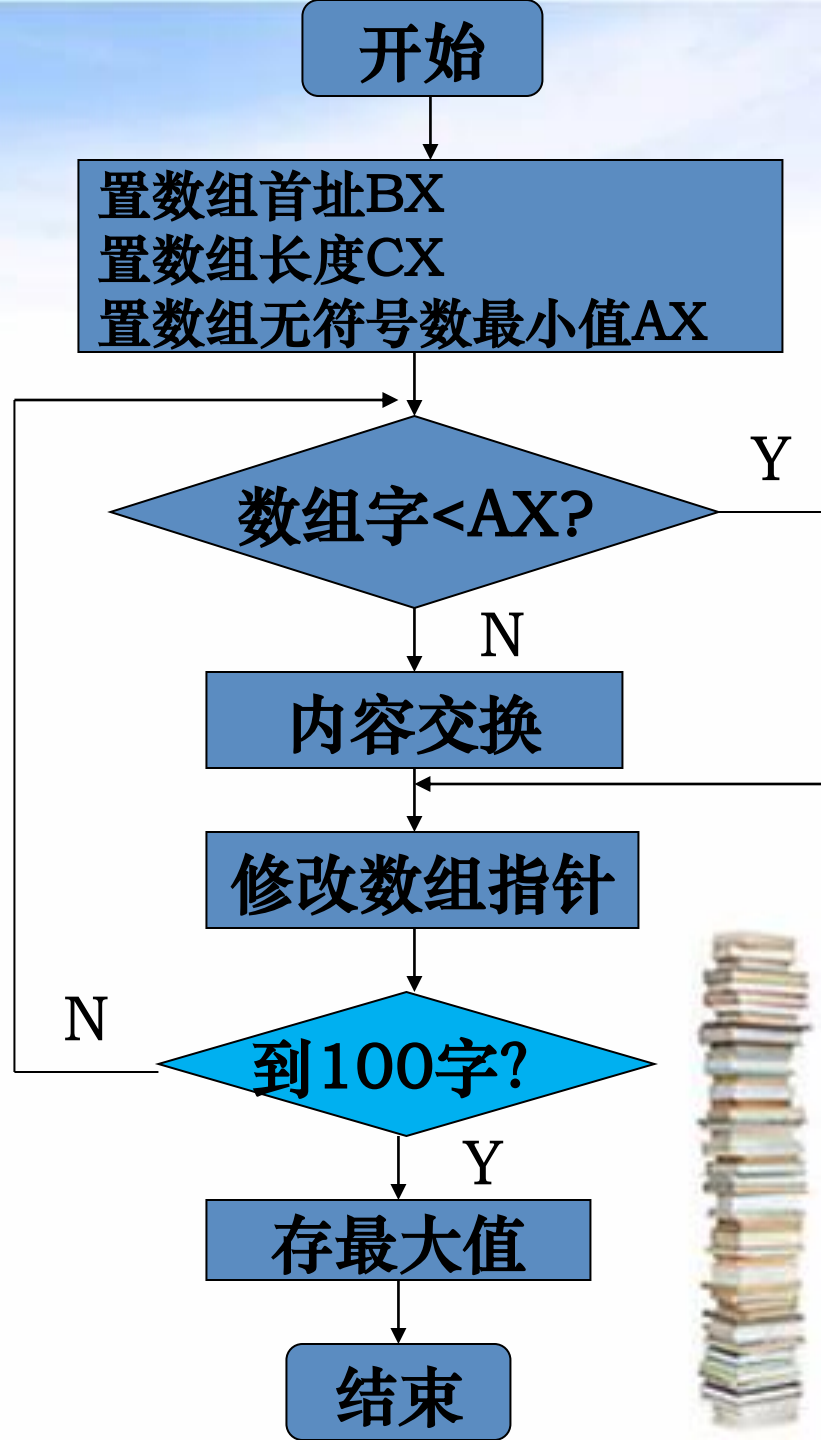
举例

例：

求无符号数组中的最大值。

已知：

数组存放在20042H开始的单元，字组长度100，最大值送存20040H开始的单元。(实验5A)



```
START:  MOV     AX, 2000H
        MOV     DS, AX
        MOV     BX, 0042H
        MOV     AX, 0
        MOV     CX, 100
NEXT1:  CMP     AX, [BX]
        JA      NEXT
        MOV     AX, [BX]
NEXT:   INC     BX
        INC     BX
        LOOP    NEXT1
        MOV     [0040H], AX
        HLT
```

循环次数已知
(计数控制)

; AX高于[BX]转到NEXT

必须采用寄存器
间址，才能实现
循环。



4.6.4 子程序

子程序：也叫过程，相对主程序而言，是一个子的程序段，它能被主程序调用，也能被其它子程序调用

一般把有公用性、重复性或有相对独立性的程序设计成子程序

- ◇把功能相对独立的程序段单独编写和调试，作为一个相对独立的模块（子程序）供其他程序使用。
- ◇子程序可以实现源程序的模块化，可简化源程序结构，提高编程效率。
- ◇主程序需要利用**CALL**指令调用子程序。
- ◇子程序需要利用**RET**指令返回主程序。



◇汇编语言中，子程序要用一对过程伪指令**PROC**和**ENDP**声明，格式如下：

过程名 **PROC** [**NEAR**/FAR]

..... ; 过程体

.....

过程名 **ENDP**

◆**NEAR**属性（段内近调用）

◆**FAR**属性（段间远调用）



一、子程序编写注意事项

- (1)子程序要利用过程定义伪指令声明。
- (2)子程序中对堆栈的压入和弹出操作要成对使用，保持堆栈的平衡。
- (3)子程序应安排在代码段的主程序之外，最好放在主程序执行终止后的位置，也可以放在主程序开始执行之前的位置。
- (4)处理好子程序与主程序间的参数传递问题。
- (5)提供必要的子程序说明信息。



二、举例

例:

主程序段

code segment

main proc far

.....

call subr1

.....

ret

main endp

子程序段

subr1 proc near

.....

ret

subr1 endp

code ends

子程序段

code segment

main proc far

.....

call subr1

.....

ret

subr1 proc near

.....

ret

subr1 endp

main endp

code ends

主程序段



例:

```
MY      SEGMENT
        ASSUME  CS:MY

MAIN    PROC    FAR
START:  PUSH    DS
        SUB     AX,AX
        PUSH    AX
        MOV     CL,04
        CALL    DP5
        MOV     BL,CL
        CALL    DP5
        ADD     AL,BL
        RET
MAIN    ENDP
```

```
DP5     PROC    NEAR
        MOV     AL,CL
        SAL     AL,1
        SAL     AL,1
        ADD     AL,CL
        MOV     CL,AL

        RET
DP5     ENDP
MY      ENDS
        END     START

A L = ?      B L = ?
C L = ?
```



例：累加数组中的元素。

主程序

```
data segment
ary dw 1,2,3,4,5,6,7,8,9,10
count dw 10
sum dw ?
data ends
code segment
main proc far
assume cs:code, ds:data
start: push ds
      sub ax, ax
      push ax
      mov ax, data
      mov ds, ax
      call proadd
      ret
main endp
```

子程序

```
proadd proc near
      push ax
      push cx
      push si
      lea si, ary
      mov cx, [count]
      xor ax, ax
next:: add ax, [si]
      add si, 2
      loop next
      mov [sum], ax
      pop si
      pop cx
      pop ax
      ret
proadd endp
code ends
end start
```



复习1、子程序的调用与返回（1）

(1) 段内直接调用与返回

格式: **CALL** 过程名

功能: 调用当前段内的子程序

操作: $SP \leftarrow SP - 2$, $[SP] \leftarrow IP$, $IP \leftarrow IP + disp$

格式: **RET**

操作: $IP \leftarrow [SP]$, $SP \leftarrow SP + 2$

子程序入口地址与
当前地址的差值

(2) 段内间接调用与返回

格式: **CALL OPRD**

操作: $SP \leftarrow SP - 2$, $[SP] \leftarrow IP$, $IP \leftarrow (OPRD)$

格式: **RET**

操作: $IP \leftarrow [SP]$, $SP \leftarrow SP + 2$

16位通用寄存器或
字存储器



复习1、子程序的调用与返回（2）

(3) 段间直接调用与返回

远过程

格式: **CALL** 过程名

功能: 调用其它代码段中的子程序

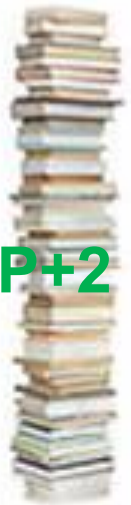
操作: $SP \leftarrow SP - 2$, $[SP] \leftarrow CS$, $SP \leftarrow SP - 2$, $[SP] \leftarrow IP$

$IP \leftarrow$ 过程入口地址的偏移量

$CS \leftarrow$ 过程入口地址的段值

格式: **RET**

操作: $IP \leftarrow [SP]$, $SP \leftarrow SP + 2$, $CS \leftarrow [SP]$, $SP \leftarrow SP + 2$



复习1、子程序的调用与返回 (3)

(4) 段间间接调用与返回

双字存储器操作数

格式: **CALL OPRD**

操作: $SP \leftarrow SP - 2$, $[SP] \leftarrow CS$, $SP \leftarrow SP - 2$, $[SP] \leftarrow IP$
 $IP \leftarrow (OPRD_{低})$, $CS \leftarrow (OPRD_{高})$

格式: **RET**

操作: $IP \leftarrow [SP]$, $SP \leftarrow SP + 2$, $CS \leftarrow [SP]$, $SP \leftarrow SP + 2$



2、子程序设计与应用应注意的问题

(1) 现场保护与恢复 { 子程序中要保护该程序中要使用的寄存器
返回时，恢复现场。后入栈的要先弹出。

例：

```
DELAYIS: PUSHF
          PUSH
```

```
CX      MOV CX, 176H ; 循环374次
```

```
LP1:    PUSHF
```

```
        POPF
```

```
        LOOP LP1      ; CX-1, CX≠0转LP
```

```
        POP CX
```

```
        POPF
```

```
        RET
```

后进先出



2、子程序设计与应用应注意的问题

(2) 参数传递——必须设计好子程序中用到了哪些寄存器或存储单元。哪些是需要调用程序提供的，哪些是结果提供给调用程序的。

(3) 子程序说明——一个子程序，可以被其它程序调用，必须提供一些说明。如：

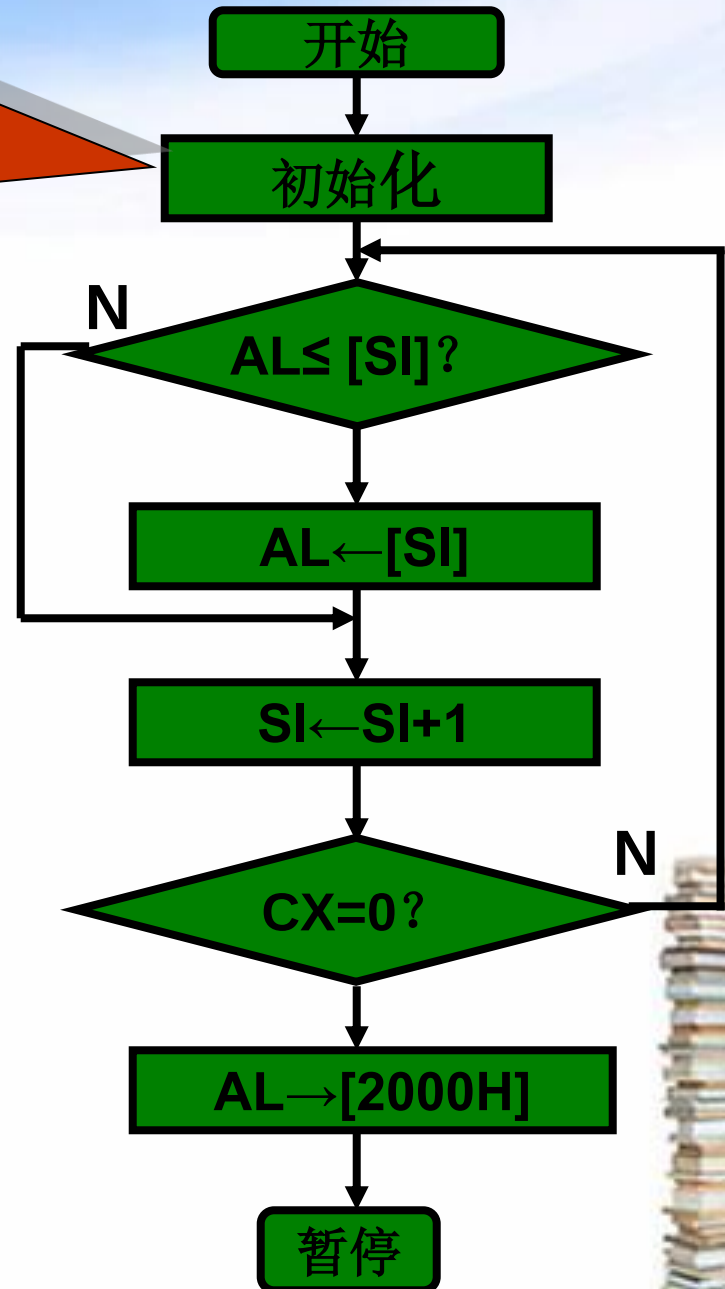
程序名、程序的功能、占用的寄存器或存储单元、入口及出口参数、嵌套了哪些子程序。



SI=2001H
循环次数CL=[SI]
最大值初值AL=00
SI+1指向数据块

子程序设计举例：

找出一个数据块中的最大数，数据块长度在（2001H）单元，数据块从（2002H）单元开始存放，把找出的最大值放到（2000H）单元，假设数据块中的数是无符号的 8 位数



MAX PROC NEAR

PUSHF

PUSH AX

PUSH CX

PUSH SI

MOV SI, 2001H

MOV CL, [SI]

INC SI

MOV AL, 00

MOV CH, 00

NEXT: CLC

CMP AL, [SI]

JC LESS

JMP NLESS

LESS: MOV AL, [SI]

NLESS: INC SI

LOOP NEXT

MOV [2000H], AL

HLT

入口信息: 2002H单元存放数据
2001H单元存放数据个数
出口信息: 最大值在2000H单元

POP SI

POP CX

POP AX

POPF

RET

ENDP MAX

4.6 查表程序

STAR: MOV BX, 2000H

MOV AL, 5

→ XLAT

HLT

[BX+AL]

MOV AH,0

ADD BX,AX

MOV AL,[BX]

执行后: AL=19H

复习换码指令

.....	
64H	
51H	
40H	
31H	
24H	
19H	
10H	
09H	
04H	
01H	2001H
00H	2000H



DATA SEGMENT

TABLE DB 0,1,8,27,64,125,216

X DB 2

Y DB 3

Z DB ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA

MAIN PROC FAR

START: PUSH DS

SUB AX,AX

PUSH AX

MOV AX,DATA

MOV DS,DATA

MOV AL,X

CALL AA

MOV CL,AL

ADD AL,AL

ADD AL,CL

MOV DL,AL

MOV AL,Y

CALL AA

ADD DL,AL

MOV Z,DL

RET

MAIN ENDP

AA

PROC

NEAR

MOV BX,OFFSET TABLE

MOV

AH,0

ADD

BX,AX

MOV

AL,[BX]

XLAT

RET

AA

ENDP

CODE

ENDS

END

START

Z = ?



The end, thank you!

